CS 320 Project Two: Summary and Reflections Report

Grand Strand Systems – Mobile Application Testing

Summary

Alignment of Testing Approach With Software Requirements

The testing approach used in Project One was closely aligned with the functional requirements of the contact, task, and appointment services. Each class contained specific constraints, such as unique identifiers, field length limits, and validation rules, and the unit tests were designed directly around those requirements. For example, in the ContactServiceTest class, the test `testDuplicateContactId()` verifies that duplicate IDs are rejected:

```
assertThrows(IllegalArgumentException.class, () ->
service.addContact(c2));
```

This directly aligns with the requirement that contact IDs must be unique. Similarly, field validation constraints were tested through constructor inputs to ensure values exceeding limits would trigger the appropriate exceptions.

In the TaskServiceTest class, tests such as `testDuplicateTaskId()` confirm that the service enforces uniqueness requirements, while update and delete tests verify the functional behaviors expected from the service layer.

The AppointmentTest class demonstrates alignment with temporal validation requirements. The helper method `futureDate()` creates valid future dates, while `pastDate()` ensures that invalid past dates are properly rejected. This directly supports the requirement that appointment dates cannot be in the past.

Because each requirement from the specification was mapped to at least one test case, the testing approach demonstrates strong alignment with the software requirements.

Effectiveness of JUnit Tests

The effectiveness of the unit tests is demonstrated through coverage of four categories: positive test cases (valid inputs), negative test cases (invalid inputs), boundary conditions, and functional operations (add, update, delete, retrieve). For example, the following assertions confirm that state changes occur correctly after operations:

```
assertEquals("Task", stored.getName());

assertEquals("Description", stored.getDescription());
```

These assertions validate both data integrity and functional correctness after each operation. Exception-based tests using `assertThrows()` confirm that validation logic behaves correctly under failure conditions, increasing confidence in the defensive programming logic implemented within constructors and service methods.

To formally measure test effectiveness, a code coverage tool such as EclEmma (built into Eclipse) was used to analyze which lines of code were exercised by the unit tests. EclEmma highlights untested lines of code in red directly within the IDE, allowing a developer to identify gaps and write additional tests to cover those paths. The coverage index achieved across the three

service classes exceeded 80%, meeting the project threshold. This high coverage percentage is a strong indicator of test effectiveness, as it confirms that the majority of code paths were executed and validated during testing. However, it is important to note that high code coverage alone does not guarantee bug free code. It is still possible to miss requirements entirely or overlook certain edge case scenarios even when coverage numbers are high. For this reason, the test suite was designed not only to maximize coverage but also to intentionally test boundary conditions and invalid inputs. According to software testing best practices, effective unit tests should validate both expected behavior and failure conditions, which this test suite accomplishes (Ammann & Offutt, 2017).

Technically Sound Code

Technical soundness was ensured through careful test design and validation logic. The use of JUnit allowed the implementation of structured test cases with clear assertions that verified expected outcomes. For example, in the Contact tests, assertions confirmed that updates to phone numbers and addresses were correctly stored after calling update methods.

Exception-handling tests confirmed that invalid input conditions produced predictable and controlled failures. For example:

```
assertThrows(IllegalArgumentException.class, () -> service.addTask(t2));
```

This verifies that invalid operations are handled safely and predictably, preventing incorrect system states from forming. Another strategy used to maintain technical soundness was validating object state after each operation. Instead of assuming that methods worked correctly, the tests explicitly checked results using getter methods. This approach reduced the likelihood of hidden defects and ensured that internal state changes were consistent with expected behavior.

Efficient Code

Efficiency considerations were addressed through the structure of both the implementation and the tests. The service classes used in-memory data structures to manage objects, allowing quick retrieval and updates by unique identifiers. The tests verified that operations could be performed without unnecessary complexity by repeatedly adding and updating records in a controlled manner.

Efficiency in testing was further achieved through the use of helper methods. For instance, the `futureDate()` helper method in the Appointment tests reduced code duplication across multiple test cases, improving readability and maintainability. Writing small, focused tests also minimized dependencies between tests and made failures easier to diagnose. Efficient test design is considered a best practice in software development because it supports rapid feedback during development cycles (Martin, 2009).

Reflection

Testing Techniques Employed

The primary testing technique used in this project was unit testing with JUnit. Unit testing focuses on verifying the correctness of individual components in isolation from the rest of the system. This approach was appropriate because each service class had clearly defined

responsibilities and validation rules. Assertions were used to compare expected and actual values, and exception testing was used to confirm that invalid conditions were handled correctly.

Another technique used was boundary testing, which involves testing values at the limits of acceptable ranges. For example, tests confirmed that IDs and field values at maximum allowed lengths were accepted, while values exceeding those limits were rejected. Boundary testing is particularly useful for identifying errors related to validation logic and input constraints.

Other Software Testing Techniques Not Used

Although unit testing and boundary testing were the primary approaches, other techniques could have been applied. Integration testing could have been used to verify interactions between services if the system included dependencies between components. For example, testing whether the appointment service correctly interacts with the contact service when associating an appointment with a contact. System testing could also have been implemented to evaluate overall application behavior from an end-user perspective.

Performance testing was not necessary for this project due to its small scale, but it would be important in larger systems where efficiency and scalability are critical. Regression testing, which verifies that existing functionality is not broken by new changes, was also not formally applied but would be valuable in a continuous development environment.

Practical Uses and Implications of Testing Techniques

Unit testing is valuable because it allows developers to detect defects early in the development lifecycle, reducing the cost and complexity of fixing errors later. Boundary testing helps ensure reliability by validating edge cases that are often sources of defects. In larger software projects, combining unit testing with integration and system testing provides comprehensive coverage that improves overall software quality. Integration testing becomes essential in enterprise systems where services communicate across APIs and databases, while performance testing is critical for applications that must handle high traffic or process large data volumes. These techniques are widely applicable across software development environments, from small mobile applications to large enterprise systems.

Caution in Testing

Employing caution during testing is essential because incomplete or incorrect tests can create false confidence in software reliability. During this project, caution was applied by verifying both valid and invalid scenarios and ensuring that tests covered edge cases. For example, in the AppointmentTest class, tests such as `testPastAppointmentDate()` were written specifically to confirm that dates in the past were rejected with the appropriate exception. Care was also taken when writing contact field validation tests to confirm that null values and oversized strings each triggered `IllegalArgumentException` as expected. This cautious approach helps prevent defects from reaching production environments and avoids the risk of tests that pass for the wrong reasons.

Limiting Bias in Testing

Bias can occur when developers test code based only on expected successful outcomes rather than potential failure conditions. To reduce bias, tests were intentionally designed to challenge the implementation with invalid data and unexpected scenarios. For example, in the

`ContactServiceTest` class, tests attempted to update contacts using non-existent IDs to confirm that the system correctly rejected those operations. Similarly, attempting to delete records that had not been added ensured that the system behaved safely under adverse conditions. Limiting bias improves the objectivity of testing and increases confidence in software quality. A developer testing their own code must be especially mindful of this, as familiarity with the implementation can lead to unconsciously avoiding scenarios that might expose flaws.

Discipline in Software Development

Discipline is a critical factor in producing high-quality software. Writing tests alongside implementation encourages developers to think carefully about requirements and edge cases before completing functionality. In this project, discipline was demonstrated by following consistent naming conventions for test methods, organizing tests by service class, and ensuring that every public method in each service had corresponding test coverage. Maintaining organized test structures and consistent validation logic contributes to long-term maintainability and reliability. Practicing disciplined development habits also helps prevent technical debt by ensuring that code remains understandable and testable over time (Martin, 2009). Avoiding shortcuts such as skipping edge case tests when the happy path already passes, is essential to upholding quality standards as a software engineering professional.

The testing process used in this project demonstrated the importance of aligning unit tests with software requirements, verifying both normal and edge case behaviors, and maintaining disciplined development practices. By applying structured testing techniques including unit testing and boundary testing, and validating system behavior through JUnit tests, confidence in the correctness and reliability of the application was significantly improved. These practices are essential for producing dependable software systems and will remain valuable throughout future software development projects.

Citations:

Ammann, P., & Offutt, J. (2017). Introduction to software testing (2nd ed.). Cambridge University Press.

Martin, R. C. (2009). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). Wiley.